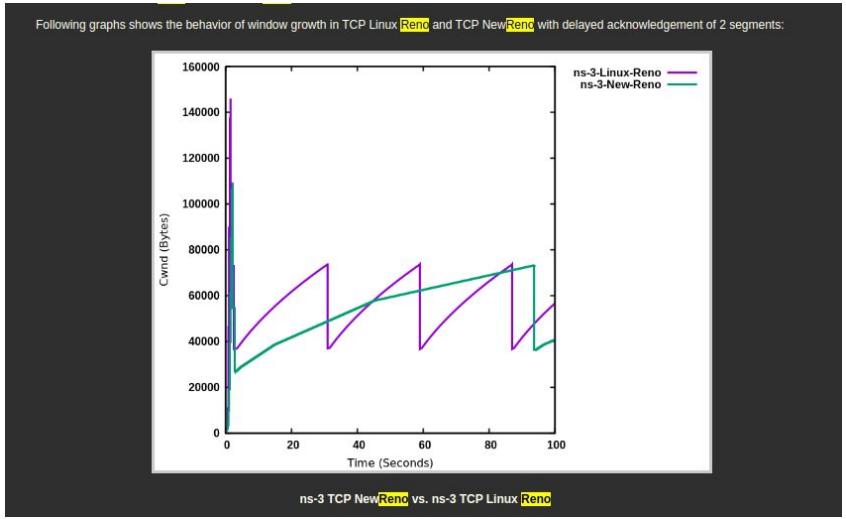
Lab 8 - Tcp Congestion Control Algorithm in NS3

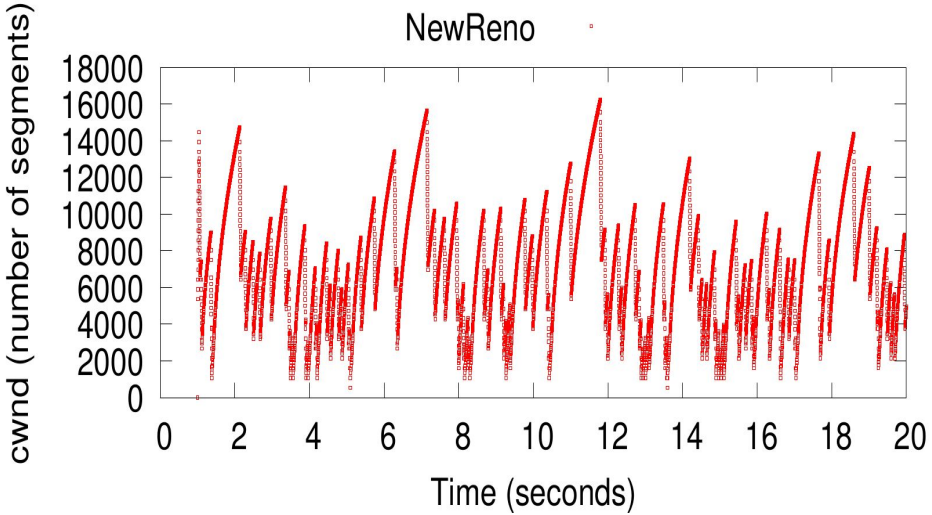
Agenda

- Tracing Congestion Window, RTT Changes with the help of ns3 callback functions.

Diving deeper into TCP

- Observing Congestion Window Changes





Callback functions (Demo_2.cc)

- Mainly used for tracking changes in network metrics.
- Ptr<Socket> **ns3TcpSocket** = Socket::CreateSocket (nodes.Get (0), TcpSocketFactory::GetTypeId ());
- A TCP socket is created on Node 0, which is used for the TCP connection in the simulation. This socket will be monitored for changes in its attributes, like congestion window size.

Callback functions (Demo_2.cc)

- The **TraceConnectWithoutContext** method is used to connect the **CongestionWindow** trace source of the **ns3TcpSocket**.
- **MakeBoundCallback** binds the **CwndChange** function to the trace source. This function will be called whenever the congestion window changes.
- The **CwndChange** function takes an **OutputStreamWrapper** (used for writing to a file), the old congestion window size, and the new size. It logs the changes to the file "**seventh.cwnd**".

Some fundamentals …

- The file paths for some fundamental classes or files are as follows:
  - /home/arindam/Desktop/Arindam Thesis/Simulation Setup/ns-allinone-3.44/ns-3.44/src/internet/model/tcp-socket-base.cc
  - /home/arindam/Desktop/Arindam Thesis/Simulation Setup/ns-allinone-3.44/ns-3.44/src/internet/model/tcp-congestion-ops.cc
- All operations that are delegated to a congestion control are contained in the class TcpCongestionOps. It mimics the structure tcp_congestion_ops of Linux, and the following operations are defined:

```
virtual std::string GetName() const;
virtual uint32_t GetSsThresh(Ptr<const TcpSocketState> tcb, uint32_t bytesInFlight);
virtual void IncreaseWindow(Ptr<TcpSocketState> tcb, uint32_t segmentsAcked);
virtual void PktsAcked(Ptr<TcpSocketState> tcb, uint32_t segmentsAcked,const Time& rtt);
virtual Ptr<TcpCongestionOps> Fork();
virtual void CwndEvent(Ptr<TcpSocketState> tcb, const TcpSocketState::TcpCaEvent_t event);
```

Some fundamentals ...

- The most interesting methods to write are **GetSsThresh** and **IncreaseWindow**. The latter is called when **TcpSocketBase** decides that it is time to increase the congestion window. Much information is available in the Transmission Control Block, and the method should increase cWnd and/or ssThresh based on the number of segments acked.
- **GetSsThresh** is called whenever the socket needs an updated value of the slow start threshold. This happens after a loss; congestion control algorithms are then asked to lower such value, and to return it.

Some fundamentals ...

- **PktsAcked** is used in case the algorithm needs timing information (such as RTT), and it is called each time an ACK is received.
- **CwndEvent** is used in case the algorithm needs the state of socket during different congestion window event.

Some fundamentals ...

- All operations that are delegated to a loss recovery are contained in the class **TcpRecoveryOps** and are given below:

```
virtual std::string GetName() const;
virtual void EnterRecovery(Ptr<const TcpSocketState> tcb, uint32_t unAckDataCount,
                           bool isSackEnabled, uint32_t dupAckCount,
                           uint32_t bytesInFlight, uint32_t lastDeliveredBytes);
virtual void DoRecovery(Ptr<const TcpSocketState> tcb, uint32_t unAckDataCount,
                        bool isSackEnabled, uint32_t dupAckCount,
                        uint32_t bytesInFlight, uint32_t lastDeliveredBytes);
virtual void ExitRecovery(Ptr<TcpSocketState> tcb, uint32_t bytesInFlight);
virtual void UpdateBytesSent(uint32_t bytesSent);
virtual Ptr<TcpRecoveryOps> Fork();
```

References

- https://github.com/nsnam/ns-3-dev-git/blob/master/src/internet/doc/tcp.rst
- https://www.nsnam.org/doxygen/dc/d64/classns3_1_1_tcp_congestion_ops.html
- https://www.nsnam.org/docs/release/3.28/doxygen/classns3_1_1_tcp_congestion_ops.html



THANK YOU

Bash Scripting

Bash scripting is writing a series of commands in a file to automate tasks in Unix/Linux environments. Bash stands for **Bourne Again SHell**, and it's one of the most commonly used shells in Linux.

- **Creating Your First Script**
- **Open a Terminal**
  On your Linux system, open a terminal window.
- **Create a New Script File**
  You can create a file with the .sh extension, though it's not mandatory.
  bash
  nano hello_world.sh
- **Write Your First Script**
  Type the following lines into your file:
  bash
  #!/bin/bash
  echo "Hello, World!"
  - #!/bin/bash: This is called a shebang. It tells the system to use the Bash shell to execute the script.
  - echo: This command prints text to the terminal.

Bash Scripting...

- **Making the Script Executable**

  Before running the script, you need to give it execute permissions else you can run it using **bash - <option> <script_name>**. (use -x option for debugging)

  chmod +x hello_world.sh

- **Running the Script**

  Now, you can run the script using:

  ./hello_world.sh

- You should see:

  Hello, World!

Bash Scripting...

Script with Command-Line Arguments

- You can pass arguments when running the script.

```
#!/bin/bash
echo "First argument: $1"
echo "Second argument: $2"
```

- Run it like this:

```
./script.sh arg1 arg2
```

- Output:

```
First argument: arg1
Second argument: arg2
```

- Try to make a simple calculator and store the result to **out.txt** ! [TASK]

Bash Scripting...

Redirecting Output to a File:

Let's say you want to list all files in a directory and save the list to a file.

```
#!/bin/bash

# List all files and save to output.txt

ls -l > output.txt
```

- **ls -l:** Lists files in long format.
- **>**: Redirects the output to output.txt. If the file already exists, it will be overwritten.

Bash Scripting...

- **Appending Output to an Existing File:**

  If you want to add the output to an existing file instead of overwriting it, use >>.

  ```
  #!/bin/bash
  # Append the current date and time to output.txt
  date >> output.txt
  ```

- Each time you run this script, the current date and time will be added to output.txt without removing the previous content.

Gnuplot: Getting Started

- **Gnuplot** is a command-line driven plotting utility that helps visualize data through various types of graphs like line plots, scatter plots, bar charts, and more. It works with data files or inline data and supports customization for professional-looking plots.
    - **Installing Gnuplot**

      On Linux:

      Most Linux distributions have Gnuplot in their repositories.

      sudo apt-get install gnuplot # For Debian/Ubuntu

      On Windows:

      Download from Gnuplot's official website.

      Follow the installation instructions

Gnuplot: Getting Started...

- Let's say you have a file called **data.txt** with the following contents:

  1 2
  2 4
  3 6
  4 8
  5 10

  Each line represents an (x, y) pair.
- **Plotting the Data:**

  At the Gnuplot prompt or in the plot.p script:
  plot 'data.txt' using 1:2 with linespoints title 'My Data'
  'data.txt': The data file to plot.
- **using 1:2:** Tells Gnuplot to use the first column as the x-axis and the second column as the y-axis.
- **with linespoints:** Draws both lines and points.
- **title 'My Data':** Adds a title to the plot legend.

Gnuplot: Getting Started...

- **Adding Labels and Titles:**

  ```
  set title "Simple Line Plot"
  set xlabel "X-Axis"
  set ylabel "Y-Axis"
  plot 'data.txt' using 1:2 with linespoints title 'Data'
  ```

  - **set title:** Sets the graph's title.
  - **set xlabel / set ylabel:** Labels the x-axis and y-axis.
- **Changing Line Styles and Colors:**

  You can change line styles using lt (line type) and lc (line color):

  ```
  plot 'data.txt' using 1:2 with lines lt 2 lc rgb "blue" title 'Blue Line'
  ```

  - **lt 2:** Changes the line type (dashed, dotted, etc.).
  - **lc rgb "blue":** Sets the line color to blue.

Gnuplot: Getting Started...

- **Saving Your Plot to a File**

  You can export your plots as images (PNG, PDF, etc.).

  - **Saving as PNG**

    ```
    set terminal pngcairo size 800,600 enhanced font 'Arial,10'
    set output 'plot.png'
    plot 'data.txt' using 1:2 with linespoints title 'Saved Plot'
    unset output
    ```

    - **set terminal pngcairo:** Sets the output format to PNG.
    - **set output 'plot.png':** Specifies the output file.
    - **set output (without arguments):** Finalizes and saves the plot.
  - **Saving as PDF**

    ```
    set terminal pdfcairo enhanced font 'Arial,10'
    set output 'plot.pdf'
    plot 'data.txt' using 1:2 with linespoints title 'PDF Plot'
    unset output
    ```

Gnuplot: Getting Started...

- **Creating a Bar Chart:**
  If you have data in animals.txt:

  ```
  Dog 10
  Cat 7
  Bird 5
  Fish 3
  ```

- **Plot a bar chart:**

  ```
  set style data histograms
  set style fill solid 1.0 border -1
  set boxwidth 0.5
  set xlabel "Animal"
  set ylabel "Count"
  plot 'animals.txt' using 2:xtic(1) title 'Animal Count'
  ```

  - **set style data histograms:** Specifies histogram style
  - **xtic(1):** Uses the first column for x-axis labels